

Tutorium 8

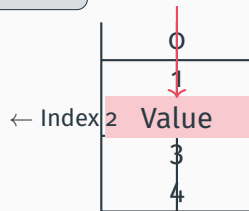
Abstrakte Datenstrukturen & Hash-Maps

Grundlagen der Praktischen Informatik

15. Juni 2026

Hash-Maps: Visualisierung

Key: „Apfel“ $\langle h(k) = f(k) \pmod{m} \rangle \rightarrow$ Index: 2



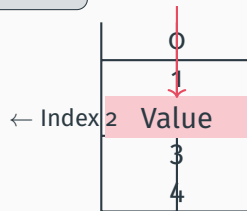
Stacks und Queues

Hash-Maps

Praxis

Hash-Maps: Visualisierung

Key: „Apfel“ $\langle h(k) = f(k) \pmod{m} \rangle \rightarrow$ Index: 2



Abstrakte Datenstrukturen (ADTs)

Stacks und Queues definieren, in welcher Reihenfolge Elemente eingefügt und entnommen werden.

Stack (LIFO)

Last In, First Out

- ▶ push: Element oben drauflegen
- ▶ pop: Oberstes Element entfernen

Beispiel: Undo-Historie, Tiefensuche.

Queue (FIFO)

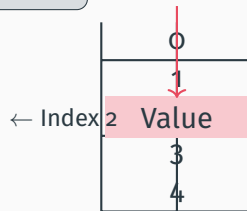
First In, First Out

- ▶ enqueue: Element hinten anstellen
- ▶ dequeue: Vorderstes Element entfernen

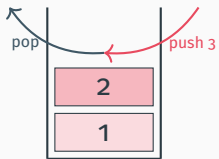
Beispiel: Warteschlangen, Breiten-suche.

Hash-Maps: Visualisierung

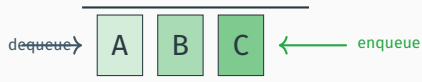
Key: „Apfel“ $\langle h(k) = f(k) \pmod{m} \rangle \rightarrow$ Index: 2



Stack (LIFO)

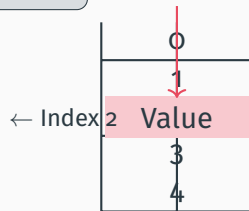


Queue (FIFO)



Hash-Maps: Visualisierung

Key: „Apfel“ $\langle h(k) = f(k) \pmod{m} \rangle \rightarrow$ Index: 2



Umsetzung in Haskell (Typklassen)

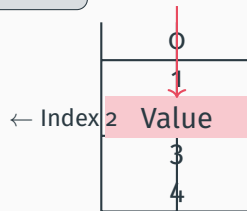
Wir können das Verhalten über Typklassen definieren.

Stack-Implementierung mit Listen

```
1  class Stack s where
2      push    :: a -> s a -> s a
3      pop     :: s a -> (a, s a)
4      sEmpty  :: s a -> Bool
5
6  instance Stack [] where
7      push    = (:)
8      pop []   = error "Stack is empty!"
9      pop (x:xs) = (x, xs)
10     sEmpty   = null
```

Hash-Maps: Visualisierung

Key: „Apfel“ $\langle h(k) = f(k) \pmod{m} \rangle \rightarrow$ Index: 2



Eine **Hash-Map** ordnet Schlüssel (Keys) auf Werte (Values) zu. Die Position in der Tabelle wird durch Hashing berechnet.

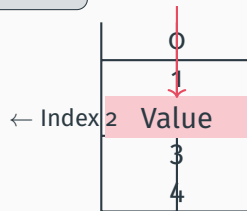
Hash-Funktion & Lastfaktor

- ▶ **Kongruenzmethode:** $h(k) = f(k) \pmod{m}$
 m wird idealerweise als Primzahl gewählt.
- ▶ **Lastfaktor (α):** Verhältnis zwischen belegten und verfügbaren Plätzen
($\alpha = \frac{n}{m}$).

Laufzeit: Im Mittel $\mathcal{O}(1)$ für Suche, Einfügen und Löschen!

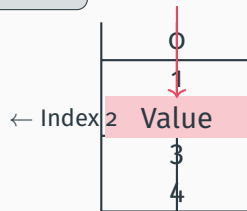
Hash-Maps: Visualisierung

Key: „Apfel“ $\langle h(k) = f(k) \pmod{m} \rangle \rightarrow$ Index: 2



Hash-Maps: Visualisierung

Key: „Apfel“ $\langle h(k) = f(k) \pmod{m} \rangle \rightarrow$ Index: 2



Kollision

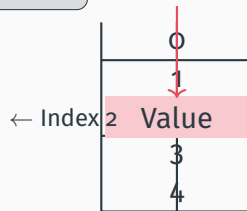
Eine Kollision liegt vor, wenn $h(k) = h(k')$ für zwei verschiedene Schlüssel $k \neq k'$.

Strategien zur Behandlung:

- ▶ **Separate Chaining:** Eine Liste / Bucket pro Tabellenfeld.
- ▶ **Offene Adressierung:** Bei Belegung weicht man auf andere Indizes aus.
 - ▶ Lineares Sondieren (+1, +2, +3 ...)
 - ▶ Quadratisches Sondieren (+1², +2², +3² ...)
 - ▶ Double Hashing (Zweite Hash-Funktion berechnet Sprungweite)

Hash-Maps: Visualisierung

Key: „Apfel“ $\rightarrow h(k) = f(k) \pmod{m} \rightarrow$ Index: 2



Live Demo



02_Abstrakte_Datenstrukturen.hs

Gemeinsame Besprechung von Hash-Maps & Klausuraufgabe (2023).

[Altklausur 2023_1 auf Moodle öffnen](#)

Hash-Maps: Visualisierung

Key: „Apfel“ $\langle h(k) = f(k) \pmod{m} \rangle \rightarrow$ Index: 2

